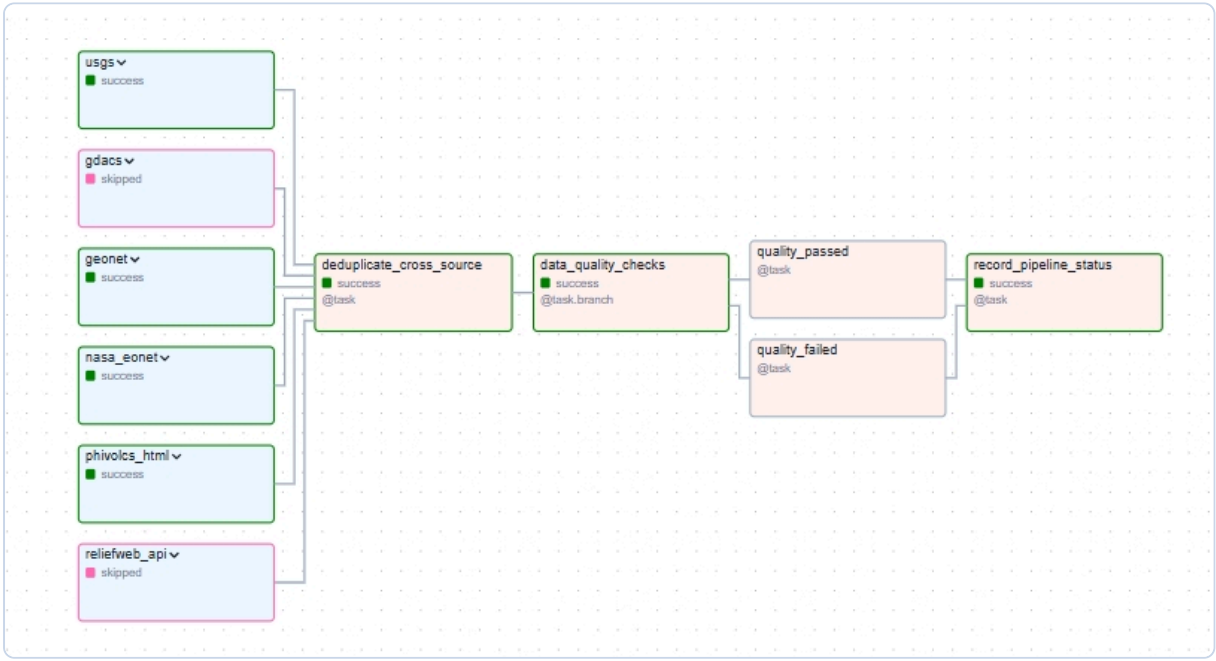
Global Disaster Intelligence Dashboard — แดชบอร์ดติดตามภัยธรรมชาติทั่วโลก

วิชา 02739452 กลุ่ม 21 · รหัสนักศึกษา 6621601123 · Repository: github.com/THEBAOxCucumber/02739452-21-6621601123 · วันที่ 23 ตุลาคม 2569

1. ภาพรวมงานแต่ละส่วน — ทำได้ครบระดับ 3 และได้โบนัส

ระบบดึงข้อมูลภัยธรรมชาติจาก **API 5 แหล่ง** (USGS, NASA EONET, GDACS, GeoNet, ReliefWeb) และ **Web Scraping 1 แหล่ง** (PHIVOLCS) ผ่าน Airflow DAG ลง PostgreSQL แล้วแสดงบนเว็บ Streamlit ภาษาไทยที่ query ฐานข้อมูลโดยตรง ปัจจุบันมี **เหตุการณ์ไม่ซ้ำ 2,957 รายการ** (เกณฑ์ ≥ 100)

ระดับ	สิ่งที่ทำและหลักฐาน	ผล
ระดับ 1 ขั้นต่ำ (30)	ทุกแหล่งมี task `extract → transform_validate → load` ลง PostgreSQL · เว็บมีตาราง และกราฟ 3 กราฟ + แผนที่	✓ ผ่าน
ระดับ 2 มาตรฐาน (20)	1 TaskGroup ต่อแหล่ง เริ่มพร้อมกัน (ขนาน) · ทุกแหล่งแปลงเป็น schema กลางตาราง `disaster_events` · UPSERT ด้วย `ON CONFLICT (source_name, source_event_id)` — รันซ้ำได้ `inserted=0` จำนวนแถวไม่เพิ่ม · เว็บมี KPI 5 การ์ด และตัวกรอง 5 แบบ (วันที่ ประเภท แหล่ง ขนาด แสดงรายการซ้ำ)	✓ ผ่าน
ระดับ 3 ท้าทาย (15)	ใช้งานจริง ≥ 4 แหล่ง · **แหล่งล่มแล้ว DAG ทำต่อ**: ทดสอบให้ USGS ล่ม → เฉพาะ `usgs.extract` เป็นสีแดง แหล่งอื่นโหลดต่อ DAG run = success (task ปลายทางใช้ `trigger_rule=all_done`) · **ตัดซ้ำข้ามแหล่ง** 234 แถวด้วย 4 กติกา (เช่น แผ่นดินไหวเวลาต่างกัน ≤ 2 นาที ห่าง ≤ 100 กม.) · **ตรวจคุณภาพ 6 ข้อ แล้วแตกเส้นทาง** ด้วย `@task.branch` → `quality_passed` / `quality_failed` และระดับแถว ข้อมูลที่ไม่ผ่าน validation แยกไปตาราง `data_quarantine` พร้อมเหตุผล	✓ ผ่าน
โบนัส (+5)	แท็บ “แหล่งข้อมูลและสถานะระบบ” แสดงสถานะ pipeline รอบล่าสุดรายแหล่ง ผลตรวจคุณภาพ และประวัติการรัน (ตาราง `pipeline_runs`)	✓ ได้
ข้อกำหนด	Web scraping PHIVOLCS ด้วย requests + BeautifulSoup (ไม่ใช้ Selenium) · ตรวจ robots.txt ทุกรอบ และ GDACS ดึงเฉพาะช่วงเวลา/อัตราที่ robots.txt กำหนด · เก็บเฉพาะข้อมูลระดับเหตุการณ์ ไม่มีข้อมูลส่วนบุคคล (PDPA) · ทุกแถวมีลิงก์ต้นทางและเวลาที่ดึงข้อมูล	✓ ครบ



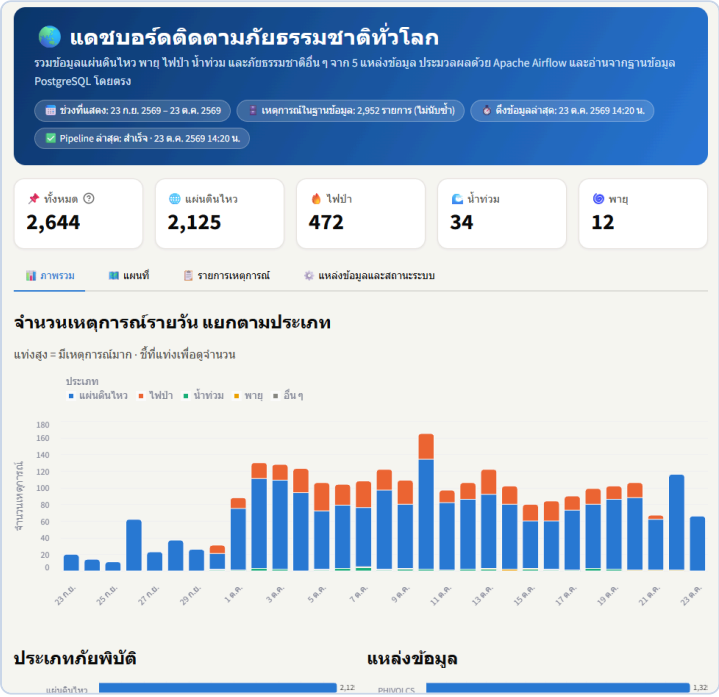
ภาพที่ 1 Graph view — 6 TaskGroup ทำงานขนาน → ตัดซ้ำข้ามแหล่ง → ตรวจคุณภาพแบบ branch → บันทึกสถานะ (GDACS ถูกข้ามเพราะอยู่นอกช่วงเวลาที่ robots.txt อนุญาต, ReliefWeb ถูกข้ามเพราะรอ appname)



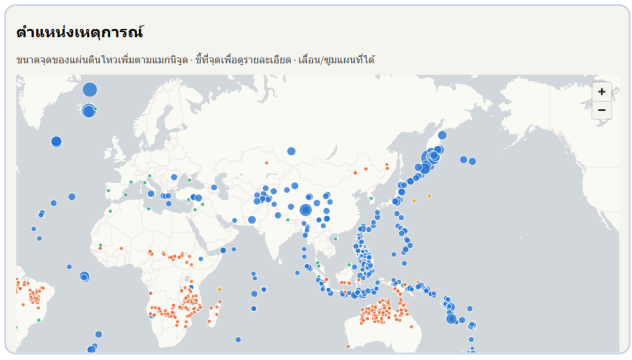
ภาพที่ 2 Grid view — ประวัติการรัน: แถว gdacs สีแดงช่วงแรก = ทดสอบแหล่งล่ม, record_pipeline_status สีแดงช่วงแรก = ปัญหาข้อ 3 ก่อนแก้, quality_failed สีเขียว = ทดสอบเส้นทางคุณภาพไม่ผ่าน

ส่วนประกอบของระบบ

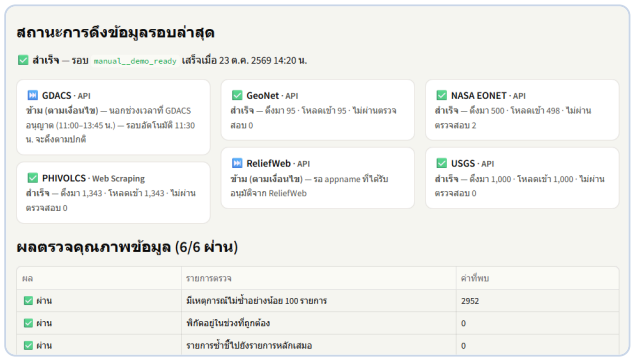
- **Extract** — 1 โมดูลต่อแหล่ง มี timeout, retry 2 ครั้ง (backoff) และเคารพ rate limit
- **Transform/Validate** — แปลงเวลาเป็น UTC (เช่น PHIVOLCS ใช้เวลาฟิลิปปินส์ UTC+8), พิกัดในช่วงถูกต้อง, ประเภทต้องเป็นภัยธรรมชาติ
- **Load** — UPSERT ลง `disaster_events` ; แถวไม่ผ่านไป `data_quarantine`
- **Dedup / Quality** — คำนวณ `duplicate_of` ใหม่ทุกรอบ (idempotent) แล้วตรวจคุณภาพ 6 ข้อ บันทึกใน `data_quality_results`
- **Web** — Streamlit อ่าน PostgreSQL ตรง: KPI, กราฟรายวัน/รายเดือน, แผนที่, ตารางค้นหาได้, สถานะ pipeline · เวลาไทย ปี พ.ศ.
- **ทดสอบ** — unit test 35 ข้อ (parser, timezone, robots.txt, rate limit, validation) และทดสอบ DAG จริงทั้งกรณีปกติ แหล่งล่ม และคุณภาพไม่ผ่าน



ภาพที่ 3 หน้าเว็บ dashboard — KPI และกราฟรายวันแยกประเภท



ภาพที่ 4 แผนที่ตำแหน่งเหตุการณ์ (ชี้ที่จุดเพื่อดูรายละเอียด)



ภาพที่ 5 หน้าสถานะ pipeline รายแหล่ง (โบนัส)

2. ปัญหาที่พบและวิธีแก้

ปัญหา / อาการ	วิธีหาสาเหตุ	วิธีแก้
1. Scraper ฝ่าฝืน robots.txt โดยไม่รู้ตัว โค้ดตรวจ robots.txt ผ่าน แต่ยังไล่ดึงหน้า 2–3 ของ ReliefWeb	อ่าน robots.txt ฉบับเต็มเองพบ `Disallow: /*&page=*` แล้วทดสอบกับ `urllib.robotparser` → พบว่า Python ไม่รองรับ wildcard `*` และ `$` จึงไม่เคยจับกฎนี้	เขียนตัวตรวจ robots.txt ตาม RFC 9309 (รองรับ wildcard, longest-match) พร้อม unit test จากกฎจริง · ภายหลังย้าย scraping ไป PHIVOLCS ซึ่งข้อมูลทั้งเดือนอยู่หน้าเดียว ดึงวันละ 1 request
2. Task load ล้มทั้งแหล่งได้ เมื่อข้อมูลเสียชิ้นเดียวกันโผล่ 2 ครั้งในรอบเดียว	ตรวจข้อมูลดิบ GDACS รายหน้าพบหน้า 2 ซ้ำกับหน้า 1 ถึง 50 รายการ → ลองเรียก `upsert_quarantine` ด้วยแถวซ้ำ จำลองได้ error `CardinalityViolation: ON CONFLICT DO UPDATE cannot affect row a second time`	ตัดรายการซ้ำตามคีย์ก่อน insert ทุกครั้ง · เปลี่ยน GDACS จากการแบ่งหน้าเป็นดึงแยกตามประเภทภัย (ได้ข้อมูลครบขึ้นและไม่ซ้ำ)

3. แหล่งเดียวล่ม ทั้ง DAG ขึ้นสีแดง ดูเหมือน DAG ล่มทั้งที่แหล่งอื่นโหลดสำเร็จ (ภาพที่ 2)	ไล่สถานะ task ใน Grid view พบว่า task สุดท้าย `record_pipeline_status` ตั้งใจ raise error เมื่อสถานะไม่ใช่ success ทำให้ Airflow ตัดสิน DAG run เป็น failed	ให้ DAG fail เฉพาะเมื่อ**ไม่มีแหล่งใดโหลดได้เลย** ส่วนแหล่งที่ล่มเป็นสีแดงเฉพาะ task ของตัวเอง และบันทึกสถานะ “สำเร็จบางส่วน”
4. GDACS ดึงได้แค่บางช่วงเวลา กด Trigger ตอนสาธิตแล้ว GDACS อาจละเมิดเงื่อนไข	อ่าน robots.txt ของ gdacs.org พบ `Request-rate: 1/60` และ `Visit-time: 0400-0645` (UTC)	เว้น 60 วินาทีต่อ request, ตั้งเวลารันอัตโนมัติ 11:30 น., นอกช่วงเวลาให้ task ข้ามตัวเอง (skipped) ไม่ทำให้ DAG ล้ม · เพิ่ม GeoNet API เป็นแหล่งที่ใช้งานได้ตลอดเวลา

3. การวางแผนและจัดลำดับงาน

หลักการจัดลำดับ

- **ทำให้ครบทั้งเส้นก่อน (end-to-end) แล้วค่อยขยาย** — แหล่งข้อมูล 1 แหล่ง → DAG → PostgreSQL → เว็บ ให้ผ่านระดับ 1 ก่อน จากนั้นเพิ่มแหล่งและฟีเจอร์ตามลำดับคะแนน ระดับ 2 → ระดับ 3 → โบนัส เพื่อให้มีงานส่งได้ทุกช่วงเวลา
- **ตรวจแหล่งข้อมูลจริงก่อนเขียนโค้ด** — เรียก API และอ่าน robots.txt ทุกแหล่งก่อน เพื่อตัดแหล่งที่ใช้ไม่ได้ออกตั้งแต่ต้น
- **เผื่อเวลาตรวจทานและตรวจตามเกณฑ์** — กันเวลาท้ายสุดไว้ทดสอบกับระบบจริงทีละข้อของเกณฑ์ (ทำให้พบปัญหาข้อ 3)

การแบ่งเวลา (23 ต.ค. 2569)

ช่วงเวลา	งาน	ผลลัพธ์
13:30–13:40	อ่านโจทย์ ทดลองเรียก API ทุกแหล่ง ตรวจ robots.txt เลือกแหล่งข้อมูล	รู้ข้อจำกัด GDACS และ ReliefWeb
13:40–13:50	ออกแบบ schema, เขียน extractor/validation/UPSERT, DAG และเว็บรุ่นแรก, unit test	รันครั้งแรกสำเร็จ (ระดับ 1–2)
13:50–14:00	ตรวจทานระบบทั้งหมด แก้ bug 6 จุด (ปัญหาข้อ 1, 2 และอื่น ๆ)	ระดับ 3 ครบ
14:00–14:20	เปลี่ยนแหล่งข้อมูลตามคำแนะนำ (ใช้ API แทน HTML), ย้าย scraping ไป PHIVOLCS, เพิ่ม GeoNet	ข้อกำหนด scraping ครบ, DAG เขียวทั้งรอบ
14:20–14:35	ปรับเว็บเป็นภาษาไทย ทดสอบด้วยภาพหน้าจอจริง, เขียน README	เว็บอ่านง่าย มีหน้าสถานะ (โบนัส)
14:35–14:50	จัดโครงสร้าง repo (dags/, web/, requirements.txt), push ขึ้น GitHub	พร้อมส่ง
14:50–15:10	ตรวจตามเกณฑ์ทีละข้อ แก้ปัญหาข้อ 3 เพิ่ม branch คุณภาพข้อมูล, จัดทำรายงาน	ระดับ 3 ไม่มีจุดเสี่ยง

งานที่ตัดออกและเหตุผล

งานที่ตัด	เหตุผล
Dynamic Task Mapping	โบนัสได้จากหน้าสถานะ pipeline แล้ว (คะแนนรวมไม่เกิน 100) และแหล่งที่แบ่งหน้าได้ (GDACS) ต้องเว้น 60 วินาทีต่อ request ตาม robots.txt จึงรันขนานแบบ map ไม่ได้
Scraping หน้าเว็บ ReliefWeb / GDACS	ReliefWeb แนะนำให้ใช้ API และห้ามไล่หน้า; GDACS โหลดรายการด้วย JavaScript และจำกัดเวลาเข้าชม → ใช้ PHIVOLCS ที่ข้อมูลอยู่ใน HTML และไม่มีข้อจำกัด
ข้อมูลจาก ReliefWeb API	ต้องรอ ReliefWeb อนุมัติ appname ทางอีเมล ซึ่งไม่ทันกำหนดส่ง → เขียนโค้ดและ test ไว้ครบ ระบบข้ามแหล่งนี้อัตโนมัติจนกว่าจะใส่ appname
แหล่ง EMSC และ NOAA	EMSC ห้าม path ของ API ใน robots.txt; NOAA มีพายุที่กำลังเกิดเพียง 1–2 ลูก ไม่คุ้มเวลา
ตัดซ้ำ ReliefWeb กับแหล่งอื่น	ReliefWeb ให้ตำแหน่งระดับประเทศ จับคู่แล้วเสี่ยงรวมเหตุการณ์ผิด จึงไม่ทำเพื่อรักษาความถูกต้อง

> ผลลัพธ์: ทำได้ครบระดับ 1–3 และโบนัส ภายในเวลาที่กำหนด โดยไม่ละเมิด robots.txt และ PDPA · วิธีติดตั้งและรันอยู่ใน README.md ของ repository